



SDC26

Technical Report

Team ToBeDefined

Swarm Drone Challenge 2026 — ILA Berlin, 11 June 2026

Abstract

This report describes the technical solution developed by Team ToBeDefined for the Swarm Drone Challenge 2026. Each flight controller runs an on-board mission engine driven by ArUco-only positioning, a domain-specific mission scripting language, and a shared command-and-control overlay that coordinates the swarm across the $20\text{ m} \times 10\text{ m}$ arena. The design prioritises drift-free, vision-relative homing over absolute world-frame steering, uses bounded closed-loop primitives for repeatable manoeuvres, and exposes a rule-based strategy layer that translates the game's scoring tiers into per-drone role and target assignments. The system is implemented across roughly 96,000 lines of code ($\approx 84,000$ in Python) and is open-sourced in line with the competition's encouragement of shared code.

1. Introduction

The Swarm Drone Challenge 2026 (SDC26) pits two teams of up to five Parrot Anafi drones against each other in a 20 m × 10 m indoor arena. Each side defends three target boxes in its home zone and attempts to flip the three enemy boxes by hovering over them for at least two seconds. The boxes carry DICT_4X4_50 ArUco markers whose encoding (leading digit – 4 for red, 3 for blue – indicates owner; trailing digit is the box ID 1–6) flips on capture. Points accrue per attempt at three tiers (1, 5, or 10) with an instant-win Special Maneuver awarded for controlling all six boxes for five continuous seconds. A match lasts ten minutes; GPS is unavailable in the indoor venue, so all positioning is derived from on-board cameras and inertial sensors.

Team ToBeDefined attacks the problem with a single, uniform software stack that runs both on each drone's flight controller (an x86 Linux companion computer flying the Anafi via Olympe) and on a lightweight central command-and-control (C2) overlay. The same code base powers the Parrot Sphinx simulator, allowing every change to be validated against the same mission scripts and the same telemetry contracts before flying real hardware.

2. System Architecture

Three processes form the runtime. **marker_mission** on port 8080 is the per-drone flight controller – it owns the Olympe connection, runs the ArUco vision loop, executes the mission scripting language, and exposes a small HTTP API for telemetry, RC, and mission control. **marker_mission_c2** on port 8090 is the fleet dashboard: it aggregates the five flight controllers, proxies operator commands, and serves the live state UI. The **strategy** module on port 8091 layers on top of the C2, providing slot tracking, manual attack triggers, and the entry point for the autonomous Commander described in Section 5.

All inter-process traffic is HTTP/JSON, and the entire fleet – five flight controllers and the operator's laptop – is joined into a single Tailscale tailnet so the operator UIs are reachable by MagicDNS name from anywhere, while still cleanly air-gapped from the public internet. Mission behaviour itself is expressed in a small text-based scripting language (the *mission DSL*) parsed and executed by the flight controller. Eighteen primitives cover discrete IMU closed-loop moves (FB_IMU, YAW_IMU), open-loop stick windows (FB_RC, UD_RC), vision homing (APPROACH, FB_BRAKE), and world-frame navigation (TO, TO_HOME). Each step compiles to a state-machine phase that emits a 20 Hz RC stream, gives vision its own thread, and writes a structured per-tick log used by the offline replay viewer for tuning.

3. Localisation

The arena provides eight pillars carrying sixteen ArUco markers – an upper marker at 4 m and a lower marker at 2 m per pillar; IDs 1–8 upper, 9–16 lower, 0.5×0.5 m, DICT_4X4_50. Each flight controller's vision loop runs OpenCV ArUco detection on the on-board camera and computes a per-marker pose with the IPPE_SQUARE planar-pose algorithm. Planar PnP returns two equally-valid solutions related by a reflection about the marker plane; we disambiguate with two independent priors.

3.1 Magnetometer prior

The arena's magnetic-north heading is calibrated once and stored in the active arena configuration. The two IPPE branches yield two candidate drone headings; the branch whose implied heading agrees with the live magnetometer reading within a configurable slack (default 35°) is accepted. This works at every altitude, including on the ground, which is critical for a clean state at takeoff.

3.2 Altimeter prior

When the drone is airborne, the Anafi's ultrasound altimeter provides a second consistency check on the vertical component of each branch. We require agreement from at least one of the two priors; if only one marker is visible and the magnetometer is uncalibrated, the fix is rejected and the previous estimate decays.

3.3 Kalman filter and anchor

Per-tick raw fixes are fused with Anafi-reported body velocities through a position Kalman filter (constant-velocity model, configurable process and measurement noise). The filter both smooths short-term measurement jitter and supplies a velocity estimate that the controller uses to damp lateral oscillation during APPROACH. When multiple reference markers are simultaneously visible, the per-marker world positions are weighted by inverse distance and fused into a single arena fix; the dominant contributor is remembered as the *anchor* across frames so that noisy single-marker fixes cannot drag the estimate in the opposite direction.

3.4 Failure modes

In practice the ArUco pipeline can still mislabel the IPPE branch in metal-rich indoor environments where the magnetometer is unreliable. We learned the hard way that an active reverse-brake wall guard acting on a mirrored single-marker fix will drive the drone *into* the wall it appears to be near, at saturated RC. The arena guard is therefore disabled by default in the current code base; the planned mitigation is a position-sanity gate that requires a fresh, low-jump fix before allowing any active correction, so wall protection can be safely re-enabled once positioning is trusted. No GPS information is used anywhere in the live positioning or control path, as required by §4.4 of the regulations – GPS is used only as an off-line ground-truth reference in the simulator during tuning.

4. Flight Control and Trajectory Planning

The Anafi exposes both a piloting (PCMD) interface for continuous roll/pitch/yaw/gaz commands and a discrete move-by interface that closes a position loop on the drone's own IMU and optical flow. Our mission engine treats these as complementary primitives. **IMU primitives** (FB_IMU, LR_IMU, UD_IMU, YAW_IMU) execute an exact bounded displacement and stop. **RC primitives** (FB_RC, UD_RC, etc.) pin a stick value for a duration and then auto-brake. **Vision primitives** (APPROACH, FB_BRAKE) home on a named ArUco marker's measured bearing and distance, which is drift-free regardless of the world-frame fix.

4.1 The canonical attack manoeuvre

All test flights converge on the following composition, generated programmatically by the C2 dashboard for the selected target slot:

```
TAKEOFF
HEIGHT 1.5
FB_IMU 4 # chain N steps to cover most of the gap
APPROACH <enemy_face> 1.2
HEIGHT 1.8
FB_IMU 1.2 # drift exactly over the box
HOOVER 3
YAW_IMU 180
FB_IMU 4 # chain back home
APPROACH <home_marker> 2.0
YAW_IMU 180
LAND
```

This pattern reflects three hard-won lessons. First, every horizontal displacement is either a bounded IMU step or a vision-homed APPROACH – never an open-loop RC cruise of non-trivial distance, because forward pitch tilts the camera downward and hides the destination marker, so the vision-tripped brake never fires and the drone reaches the timeout at full stick. Second, the over-the-box translation is a bounded FB_IMU sized to the APPROACH stand-off, so even with a noisy fix the drone stops above the target rather than continuing into the wall behind it. Third, the closing 180° rotation uses the IMU-closed-loop YAW_IMU rather than a yaw-rate stick; the RC variant rolled off in timing tests and sent the next leg sideways.

4.2 Safety layers

Several overlays guard the runtime regardless of which script is loaded. A hard altitude ceiling clamps the gaz channel above a configurable height. A C2 watchdog forces an automatic land if the operator's heartbeat goes silent for longer than the regulation's two-second threshold (§3.3). A master kill-all command lands the entire fleet from a single key-press and is the responsibility of the team's Safety Officer per §2.2. Each drone in flight is shrouded in a spherical protective cage as recommended in §3.2. Crucially, no script step can bypass the kill-switch path, and the kill-switch is wired both physically and virtually so it remains actionable if any one component fails.

5. Swarm Strategy

Strategic decisions are made by a small **Commander** module that runs as part of the strategy server. It is deliberately rule-based and deterministic: each tick it inspects every drone's status (connection, battery, magnetometer calibration, current mission phase, position) and the live slot map (the holder of each of the six boxes, and how recently we have observed it), and produces a (role, target) assignment per drone that maximises a transparent utility function:

```
U(d, t) = value(t)
  - w_time · eta(d → t)
  - w_risk · risk(t)
  + w_ready · readiness(d)
  + w_hold · [t is the drone's current task]
  - w_switch · [d is mid-task and t differs]
```

The *value* term encodes the regulations' scoring tiers directly. Recapturing an enemy-held box inside our own home zone is worth zero (the anti-farming rule of §1.4.3) and is filtered out of the task set entirely. A baseline capture is worth one game point. A capture in which every team drone is already outside the home zone at the moment of trigger is worth five and adds a coordination bonus. A pair of captures by two different drones inside one second is worth ten and is pursued opportunistically when two drones are simultaneously close to two different enemy boxes. The Special Maneuver – controlling all six boxes for at least five seconds – is treated as a terminal value and short-circuits role selection when it is within reach.

5.1 Role set and asymmetric attack

The Commander assigns one of four roles per drone: **attacker** (drive a capture or recapture), **scout** (refresh stale slots in the enemy zone so the slot map remains current), **defender** (re-flip a recently-lost home box), and **return/idle** (low battery, lost link, or no useful work). The attack set is asymmetric per team – red drones may only target boxes 4–6 (the blue zone) and blue drones boxes 1–3 – and the Commander rejects any operator override that violates the constraint. A staggered take-off schedule and per-slot airspace deconfliction prevent two drones from arriving at the same box.

5.2 Open source and reproducibility

The entire stack is open-sourced at github.com/otiedemann/sdc-tobe: approximately 96,000 lines of code, of which 84,000 are Python. The mission DSL parser ships with a test harness; the strategy module, the flight controller, and the C2 dashboard share a single configuration file and a single venv; and every flight is recorded into a per-tick CSV plus an annotated MP4 that the in-browser replay viewer plays back synchronously.

6. Conclusion

Our design philosophy across every layer of the stack is the same: prefer drift-free, vision-relative homing and bounded closed-loop motion over absolute world-frame steering and open-loop cruises. This choice is forced by the realities of ArUco-only indoor positioning, and it is what kept us flying safely through the bug hunts that preceded this report. The remaining work – a position-sanity gate that lets the wall guard be safely re-enabled, full Commander auto-dispatch, and a defender role tuned to the recapture window – is scoped and queued, and will be delivered in time for the final at ILA Berlin on 11 June 2026.